

ALBUM MOSAIC

Project Planning & Build Guide

A Spotify-connected, customizable personal music profile built for full-stack and AWS portfolio experience

Working title	Album Mosaic (rename at any time)
Primary goal	Build a polished portfolio application that demonstrates React/Next.js, TypeScript, Python/FastAPI, PostgreSQL, Docker, AWS, testing, security, and CI/CD.
Core interaction	Users arrange square album covers into a custom or preset shape that represents them, then share a read-only public profile.
Document version	v0.1 — September 30, 2026



Mobile concept: shared profile, album detail, and in-app profile discovery.

Contents

1. Product vision and objectives
2. Key product decisions and Spotify constraints
3. Scope: MVP and stretch goals
4. Technology stack
5. System architecture
6. Data model
7. API design and permissions
8. Frontend experience and component plan
9. Backend service plan
10. Step-by-step build roadmap
11. Testing strategy
12. Security and privacy
13. AWS deployment architecture
14. CI/CD and Infrastructure as Code
15. Observability and cost control
16. Milestones and schedule
17. Risks and mitigations
18. Definition of done
19. Resume and interview outcomes
20. Immediate next actions
21. Official references

1. Product vision and objectives

Album Mosaic is a personal music-profile application that connects to a user's Spotify account and lets the user design a visual identity around albums they choose from their saved library. The signature feature is a grid-based mosaic builder: each square tile is an album cover, and the tiles combine into a preset or custom shape such as a heart, star, music note, initials, icon, or freeform design.

Primary user journeys

User	Journey	Value
Profile owner	Connect Spotify → choose albums → design mosaic → customize profile → publish/share.	Creative identity plus a technical showcase of OAuth, CRUD, layout persistence, and cloud deployment.
Friend / visitor	Open shared URL → view read-only profile → tap an album → open it in Spotify.	No account required for a public profile; fast mobile-first experience.
Registered user	Search Album Mosaic profiles by username/display name → open public profiles.	Social discovery is owned by this application, not by Spotify user search.

Project objectives

- Build a real end-to-end system where each technology has a meaningful responsibility.
- Demonstrate OAuth 2.0 integration, API design, database modeling, responsive UI, authorization, testing, containerization, AWS networking, deployment, and CI/CD.
- Keep the MVP small enough to finish while preserving clear extension points for additional features.
- Produce a public deployment, readable README, architecture diagram, automated tests, and concise resume bullets.

2. Key product decisions and Spotify constraints

Decision A — Featured Albums, not “Top Albums”

Spotify exposes top artists and top tracks, but not a top-albums ranking. For this project, the mosaic should use albums explicitly chosen by the user from their saved Spotify library. This avoids inventing a derived listenership metric and keeps the product aligned with Spotify's current Developer Policy. [2][3][6]

Decision B — Profile search belongs to Album Mosaic

Spotify's Search endpoint searches catalog objects such as albums, artists, playlists, tracks, shows, episodes, and audiobooks — not Spotify users. Album Mosaic should index its own public profiles in PostgreSQL and search them by app username/display name. [4]

Decision C — Link accounts with Spotify account_id

The current-user profile endpoint returns `account_id`, described by Spotify as a public, immutable, pseudoanonymous identifier intended for account linking. Store it as the external identity key for the

connected account. [5][7]

Development-mode constraint

New Spotify apps start in Development Mode. Spotify currently documents a maximum of five authenticated users for a Development Mode app, with the app owner requiring Premium. This is appropriate for a portfolio/beta project but limits broad public sign-up unless Spotify grants Extended Quota Mode. [1]

MVP Spotify scopes

user-read-private	Read the connected user's own profile information.
user-library-read	Read the user's saved albums so they can select Featured Albums.
Optional later: user-top-read	Display Spotify-provided top artists/top tracks without creating new derived metrics.
Optional later: user-read-currently-playing	Display "Now Playing" if it adds value and policy requirements are satisfied.

3. Scope: MVP and stretch goals

MVP — must finish

- Spotify OAuth login and account linking.
- Create/edit one Album Mosaic profile: username, display name, bio, avatar from Spotify, visibility, theme preset.
- Browse the connected user's saved albums and add/remove Featured Albums.
- Grid-based album mosaic with drag/drop tile placement plus at least four presets (heart, star, music note, initials/custom blank grid).
- Persist mosaic tile coordinates in PostgreSQL.
- Public read-only profile at /@username that works on mobile and desktop.
- Tap/click an album tile to see album/artist information and open it in Spotify.
- In-app profile search over public Album Mosaic users.
- Dockerized local development and deployed AWS environment.
- Automated backend and frontend tests plus CI checks.

Stretch — only after MVP is deployed

- Multiple saved mosaics per profile and an active mosaic selector.
- Theme designer, background gradients, typography choices, and section reordering.
- Spotify-provided Top Artists / Top Tracks sections.
- Follow/bookmark other Album Mosaic profiles (your own social graph).
- Profile analytics limited to your own app events (e.g., public page views), not derived Spotify listenership metrics.
- OpenGraph/social preview image generation for shared profile links.

- Infrastructure fully provisioned by Terraform and preview deployments per pull request.

4. Technology stack

Layer	Technology	Responsibility	Resume signal
Frontend	React + Next.js + TypeScript	App Router pages, responsive UI, editor interactions, public profile, API client.	Modern React, SSR/CSR decisions, typed frontend.
Styling / UI	Tailwind CSS + optional dnd-kit	Responsive design, design tokens, accessible drag/drop mosaic editor.	Component system and interaction design.
Backend	Python + FastAPI	OAuth callbacks, application auth, REST API, authorization, Spotify client, profile/mosaic logic.	Typed Python APIs, OpenAPI, dependency injection.
Persistence	PostgreSQL + SQLAlchemy 2 + Alembic	Users, profiles, sessions, selected album IDs, mosaic layout, search.	Relational modeling, async ORM, migrations.
Local platform	Docker + Docker Compose	Reproducible frontend/backend/database environment.	Containers and service orchestration.
AWS frontend	Amplify Hosting	Deploy Next.js frontend. AWS documents Next.js SSR support through Next.js 15. [8]	Managed web hosting and deployment.
AWS backend	ECR + ECS/Fargate + ALB	Store FastAPI image; run containerized API without managing EC2 hosts. [9][10]	Containers, load balancing, IAM, networking.
AWS database	RDS for PostgreSQL	Managed PostgreSQL with private network access. [11][12]	Managed relational database operations.
Secrets / logs	Secrets Manager + CloudWatch	Protect credentials/tokens; centralize application/container logs. [13]	Cloud security and observability.
CI/CD	GitHub Actions	Lint, test, build, image push, deployment.	Automated delivery workflow.
IaC	Terraform (phase 2)	Provision VPC, ECS, RDS, IAM, ALB, secrets references.	Infrastructure as Code.

5. System architecture

Architecture principle

Keep Spotify credentials and token refresh on the FastAPI server. The browser should interact with your application API and receive only the profile data it needs; do not expose Spotify refresh tokens to Next.js or store them in localStorage.

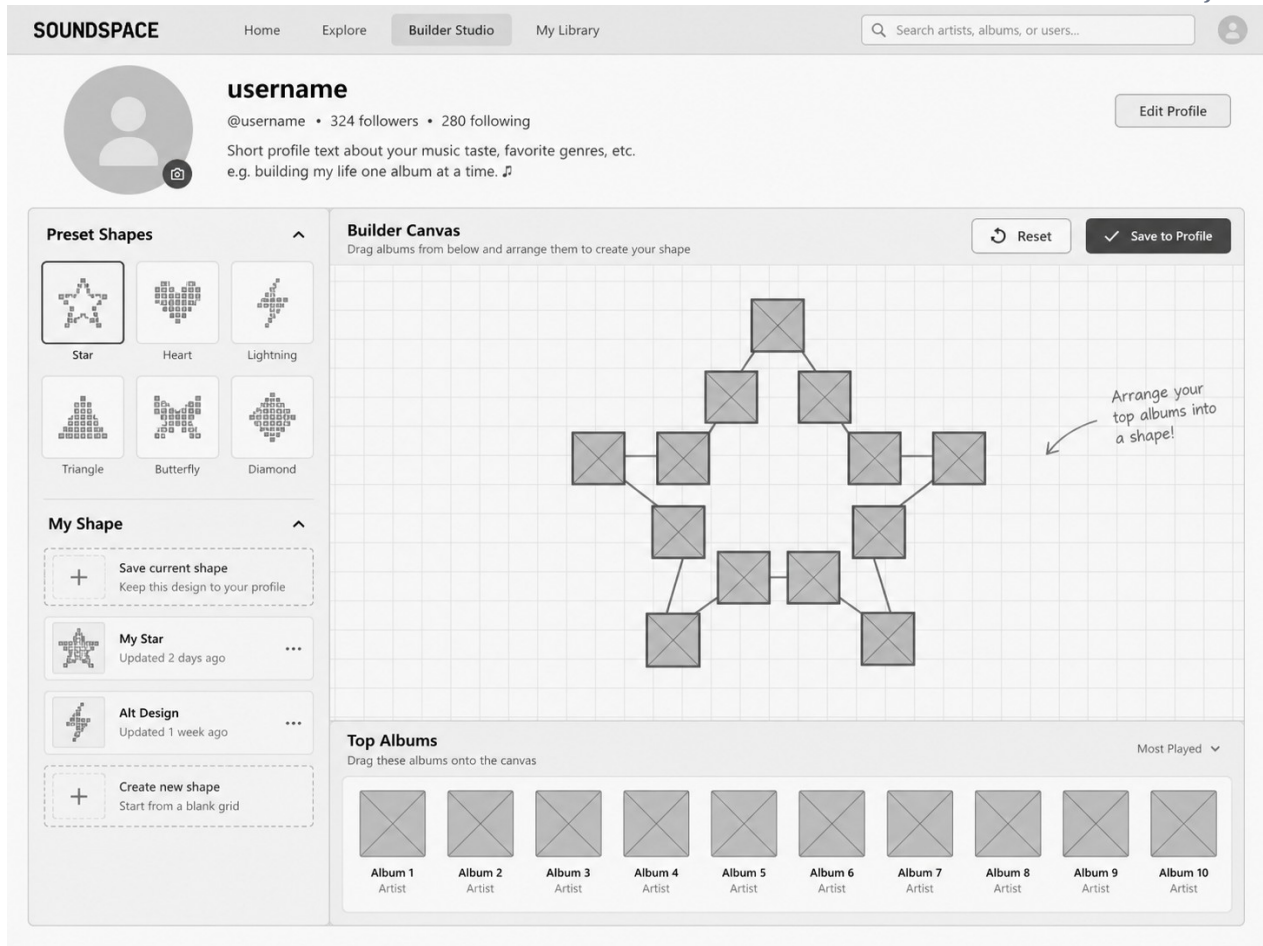
Browser / Next.js → FastAPI REST API → PostgreSQL
 **Spotify Web API**

Request flow: owner connects Spotify

1. Next.js sends the browser to GET /api/auth/spotify/start.
2. FastAPI creates OAuth state, stores temporary state, then redirects to Spotify Authorization Code flow.
3. Spotify redirects to FastAPI /api/auth/spotify/callback.
4. FastAPI validates state, exchanges the code for tokens, calls GET /me, and upserts the local user using Spotify account_id.
5. FastAPI stores the Spotify refresh token server-side (encrypted or protected by application-level encryption/KMS in production) and creates an application session.
6. FastAPI sets an HttpOnly, Secure, SameSite cookie for the Album Mosaic session and redirects the browser to the dashboard.

Request flow: friend opens a shared profile

7. Visitor opens https://yourdomain.com/@alex.
8. Next.js requests GET /api/profiles/alex/public.
9. FastAPI loads only a public profile plus mosaic layout from PostgreSQL.
10. For Spotify album metadata that must be refreshed, FastAPI resolves album IDs through Spotify and returns a normalized public DTO.
11. No write endpoints are available to the visitor; the UI renders a read-only mobile-first profile.



Desktop editor concept: preset picker + freeform grid builder + album tray.

6. Data model

Recommended tables

Table	Important fields	Purpose
users	id UUID PK; spotify_account_id UNIQUE; created_at; updated_at	Stable local identity linked to Spotify account_id.
spotify_connections	user_id FK; encrypted_refresh_token; scopes; connected_at	Server-side Spotify authorization state. Never returned to the frontend.
sessions	id/hash PK; user_id FK; expires_at; created_at	Opaque application sessions for HttpOnly cookies.
profiles	user_id FK; username UNIQUE; display_name; bio; visibility; theme JSONB	Application-owned public profile data.
mosaics	id UUID PK; profile_id FK; name; preset_key; grid_width; grid_height; is_active	One saved mosaic design.
mosaic_tiles	mosaic_id FK; spotify_album_id; x; y; rotation; z_index	Tile placement. Store only what is necessary to reconstruct the mosaic.

Table	Important fields	Purpose
profile_search (optional view)	profile_id; username; display_name	Searchable projection of public profiles; begin with ILIKE, later add pg_trgm if useful.

Mosaic tile example

```
{
  "spotifyAlbumId": "...",
  "x": 4,
  "y": 2,
  "rotation": 0,
  "zIndex": 1
}
```

Storage rule

Persist the application's profile configuration and the Spotify identifiers needed to operate it. Refresh Spotify-sourced metadata/artwork when practical rather than treating your database as a permanent copy of Spotify's catalog.

7. API design and permissions

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Health check for local/AWS load balancer.
GET	/api/auth/spotify/start	Public	Start OAuth; redirect to Spotify.
GET	/api/auth/spotify/callback	Public callback	Validate state, exchange code, create app session.
POST	/api/auth/logout	Authenticated	Delete/expire app session.
GET	/api/me	Owner	Current Album Mosaic user/profile.
PATCH	/api/me/profile	Owner	Edit username, bio, visibility, theme.
GET	/api/me/albums	Owner	Read saved Spotify albums for the selection tray.
GET	/api/me/mosaics	Owner	List saved mosaics.
POST	/api/me/mosaics	Owner	Create blank or preset mosaic.
PUT	/api/me/mosaics/{id}	Owner	Save complete tile layout.
POST	/api/me/mosaics/{id}/activate	Owner	Set the active public mosaic.
GET	/api/profiles/{username}/public	Public if visible	Read-only profile DTO.
GET	/api/profiles/search?q=	Public or auth-only	Search your app's public profiles.

Authorization rules

- Public visitor: may read only profiles with visibility = public.
- Profile owner: may update only records tied to the current session's user_id.
- Never trust a user_id sent from the frontend; derive the acting user from the server-side session.
- Use FastAPI dependencies for current_user and owner checks so authorization logic is centralized. FastAPI's dependency system is designed for shared auth/security logic and is represented in generated OpenAPI docs. [14]
- Return a minimal public DTO; exclude email, Spotify refresh token, internal IDs, connection metadata, and any private fields.

8. Frontend experience and component plan

Routes

/	Landing page and explanation of the project.
/connect	Connect Spotify call-to-action and OAuth error states.
/dashboard	Owner dashboard and profile status.
/builder	Mosaic editor: presets, grid canvas, album tray, save/publish.
/settings/profile	Username, bio, visibility, theme.
/discover	Search public Album Mosaic profiles in PostgreSQL.
/@{username}	Public read-only profile; optimized for mobile sharing.

Core React components

- ProfileHeader — avatar, display name, @username, bio, share button.
- MosaicCanvas — responsive CSS grid that renders tiles from persisted coordinates.
- MosaicEditor — drag/drop editing mode; keyboard-accessible controls as a fallback to drag/drop.
- AlbumTile — square cover, focus state, selection state, optional detail sheet.
- AlbumPicker — paginated/searchable tray of saved albums.
- PresetPicker — heart/star/music-note/custom templates represented as coordinate maps.
- AlbumDetailSheet — mobile bottom sheet with album/artist and “Open in Spotify.”
- ProfileSearch — debounced input + public profile cards.
- VisibilityControl — public/private setting with plain-language explanation.

Responsive behavior

- Mobile public profile: single-column, profile header → mosaic → Featured Albums; tap tile for details.
- Desktop public profile: wider profile header with the mosaic centered as the visual hero.
- Mobile builder: use an edit mode with one selected tile at a time; keep the album tray in a bottom sheet.
- Desktop builder: side preset rail + central grid + bottom album tray, matching the wireframe concept.
- Use CSS Grid for rendering the final mosaic; store coordinates independently of device pixels so the design scales cleanly.

9. Backend service plan

auth service	OAuth state, callback processing, session creation, logout.
spotify client	Token refresh, GET /me, saved albums, optional top items; retry/429 handling.
profile service	Profile validation, username uniqueness, visibility rules.
mosaic service	Preset expansion, tile validation, ownership checks, active mosaic selection.
search service	Query public local profiles; pagination and basic ranking.
repository layer	SQLAlchemy queries isolated from HTTP route handlers.
schemas	Pydantic request/response DTOs. FastAPI exposes OpenAPI/Swagger automatically. [15]

Suggested backend layout

```

backend/app/
  api/routes/
  core/config.py
  db/session.py
  models/
  schemas/
  repositories/
  services/spotify.py
  services/auth.py
  services/mosaic.py
  main.py

```

10. Step-by-step build roadmap

Phase 0 — Repository and development environment

Outcome

A clean monorepo boots locally and establishes quality gates before product work begins.

Build tasks

- Create GitHub repository with frontend/, backend/, infrastructure/, docs/.
- Create Next.js App Router project with TypeScript, ESLint, Tailwind, and a simple design token file. Next.js documents the App Router as the current file-system router for modern React features. [16]
- Create FastAPI project; expose GET /api/health and verify /docs.
- Add .editorconfig, .gitignore, root README, environment-variable examples, and commit hooks if desired.
- Add Docker Compose with PostgreSQL first; containerize frontend/backend later in Phase 8.

Acceptance criteria

- npm run lint and frontend type-check pass.
- FastAPI starts and /api/health returns 200.
- Local PostgreSQL is reachable.
- README has one-command startup instructions.

Phase 1 — Spotify OAuth vertical slice

Outcome

A user can connect Spotify and the UI displays their Spotify display name and avatar.

Build tasks

- Create Spotify developer app and configure development redirect URI.
- Implement /auth/spotify/start with cryptographically random state.
- Implement callback and authorization-code exchange in FastAPI.
- Call GET /me and use account_id as the durable account-linking identifier. [5]
- Create a temporary application session and redirect to /dashboard.
- Handle OAuth denial, invalid state, expired/failed token exchange, 401/403/429 responses.

Acceptance criteria

- Spotify refresh token never appears in browser storage.
- Dashboard renders real connected-user profile data.
- OAuth error states are visible and recoverable.
- One integration test covers callback/state validation with Spotify calls mocked.

Phase 2 — PostgreSQL schema and migrations

Outcome

Connected users persist locally and database changes are versioned.

Build tasks

- Add SQLAlchemy 2 models and async PostgreSQL connection (asyncpg). SQLAlchemy supports asyncio APIs for ORM/Core usage. [17]
- Add Alembic; create initial migration for users, spotify_connections, sessions, profiles. Alembic is the SQLAlchemy migration tool. [18]
- Add unique username and spotify_account_id constraints.
- Implement repository functions and transaction boundaries.
- Seed one local test user/profile fixture.

Acceptance criteria

- alembic upgrade head builds a blank database.
- OAuth callback upserts the same user on repeat login.
- Repository tests run against a disposable test database.

Phase 3 — Profile CRUD and read-only sharing**Outcome**

An owner can edit a profile and share a public URL that a logged-out visitor can read.

Build tasks

- Implement profile edit form for username, display name, bio, visibility, theme.
- Create PATCH /api/me/profile with validation and uniqueness errors.
- Create GET /api/profiles/{username}/public with public DTO.
- Implement /@username Next.js route and mobile-first shared profile shell.
- Add Share button using Web Share API with copy-link fallback.

Acceptance criteria

- Private profiles are not exposed through the public endpoint.
- Logged-out browser can open a public shared link.
- Non-owner write attempts fail server-side.
- Mobile layout works at common 360–430px widths.

Phase 4 — Saved albums and Featured Albums**Outcome**

The owner can browse their saved Spotify albums and choose the albums used in the mosaic.

Build tasks

- Request user-library-read and call GET /me/albums, which Spotify documents for saved albums. [3]
- Normalize Spotify album responses in the backend instead of leaking raw provider payloads into UI components.
- Create AlbumPicker with pagination/load-more.
- Persist selected Spotify album IDs only as necessary to reconstruct the design.
- Build album detail component with a Spotify link.

Acceptance criteria

- At least 20 saved albums load from the connected account.
- User can add/remove albums from the working mosaic set.

- API handles empty library, expired access token, and quota/rate-limit errors.

Phase 5 — Mosaic builder

Outcome

The project's signature feature works: albums form a custom or preset shape and the layout persists.

Build tasks

- Represent every preset as a list of grid coordinates.
- Create heart, star, music-note, and blank/custom presets.
- Build MosaicCanvas with CSS Grid and square aspect-ratio tiles.
- Add drag/drop or click-to-place editing; maintain keyboard-accessible placement controls.
- Save all tile coordinates to PUT /api/me/mosaics/{id} in one transaction.
- Add Reset, Undo (client-side), Save, and Set Active actions.
- Validate coordinate uniqueness, grid bounds, tile count, and album ownership on the server.

Acceptance criteria

- Refresh preserves the exact layout.
- Same saved layout scales on phone and desktop.
- Invalid/duplicate coordinates are rejected by the backend.
- At least four presets can be applied with one click.

Phase 6 — Discover profiles

Outcome

Users can search profiles created inside Album Mosaic without depending on Spotify user search.

Build tasks

- Implement GET /api/profiles/search?q=&limit=&cursor=. Spotify's catalog search does not support user objects, so discovery remains local. [4]
- Start with case-insensitive username/display_name search.
- Return only public profiles with small preview metadata and mosaic thumbnail coordinates.
- Add debounced search UI and empty/loading/error states.
- Optional: introduce pg_trgm index if real query volume justifies it.

Acceptance criteria

- Search never returns private profiles.
- Result links open the correct /@username profile.
- Search query is parameterized through SQLAlchemy, not string-concatenated SQL.

Phase 7 — Testing, accessibility, and security pass

Outcome

Core flows are covered by automated tests and obvious security problems are removed before deployment.

Build tasks

- Backend: pytest unit tests for services + API tests for auth/profile/mosaic/search.
- Mock Spotify HTTP calls; do not make live Spotify requests in CI.
- Frontend: Vitest/React Testing Library for forms, mosaic rendering, read-only state, search states.
- Add Playwright smoke test: login can be mocked, edit profile, save mosaic, view public route.
- Run axe or equivalent accessibility checks on main routes.
- Add structured error responses and centralized logging with secrets redacted.

Acceptance criteria

- Critical authorization paths have tests.
- No test fixture contains real Spotify tokens or personal credentials.
- Keyboard user can navigate public profile and essential editor controls.
- CI fails on lint, type, or test failure.

Phase 8 — Docker and production containers**Outcome**

Local and cloud runtime behavior become reproducible.

Build tasks

- Add multi-stage Dockerfile for FastAPI and health check.
- Optionally containerize Next.js locally; cloud frontend can still use Amplify.
- Update docker-compose.yml for frontend/backend/postgres with named volume.
- Use non-root runtime user where practical.
- Keep secrets out of Docker images; inject environment variables at runtime.

Acceptance criteria

- docker compose up starts the complete local app.
- Backend image runs independently and exposes /api/health.
- No .env or secret is copied into an image layer.

Phase 9 — AWS deployment**Outcome**

A public HTTPS environment runs the real system on AWS.

Build tasks

- Deploy Next.js to Amplify Hosting. AWS documents SSR hosting for supported Next.js versions. [8]
- Create ECR repository and push the FastAPI image. AWS documents docker push workflows for private ECR repositories. [19]
- Create ECS/Fargate service behind an Application Load Balancer. [9]
- Create RDS PostgreSQL in private subnets; AWS's RDS PostgreSQL tutorial describes private DB instances as a best-practice pattern. [12]
- Store DB credentials and Spotify application secrets in Secrets Manager; grant least-privilege task-role access. Secrets Manager is intended for encrypted credentials, OAuth tokens, and similar secrets. [13][20]
- Create CloudWatch log groups and ECS awslogs configuration.
- Configure CORS, cookies, allowed hosts, callback URLs, and HTTPS production domains.

Acceptance criteria

- Public profile loads over HTTPS.
- ALB health check is green.
- RDS has no public internet exposure.
- Secrets are not stored in source control or ECS task definitions as plaintext values.
- CloudWatch shows application logs without sensitive token values.

Phase 10 — CI/CD and Terraform

Outcome

Commits are tested automatically and infrastructure can be recreated from code.

Build tasks

- GitHub Actions PR workflow: frontend lint/type/test + backend lint/test.
- Main-branch workflow: build FastAPI image, authenticate to AWS using OIDC, push to ECR, update ECS service.
- Let Amplify deploy the frontend from the main branch, or control it through a separate pipeline.
- Add Terraform for VPC/subnets/security groups, ALB, ECS cluster/service/task, RDS, IAM, log groups, and references to Secrets Manager.
- Use remote Terraform state only when you are ready to manage it safely; document teardown.

Acceptance criteria

- A clean clone can provision/reproduce the environment with documented prerequisites.
- Main branch deploy requires passing tests.
- AWS credentials are not long-lived GitHub repository secrets if OIDC is used.
- terraform destroy / cleanup procedure is documented for cost control.

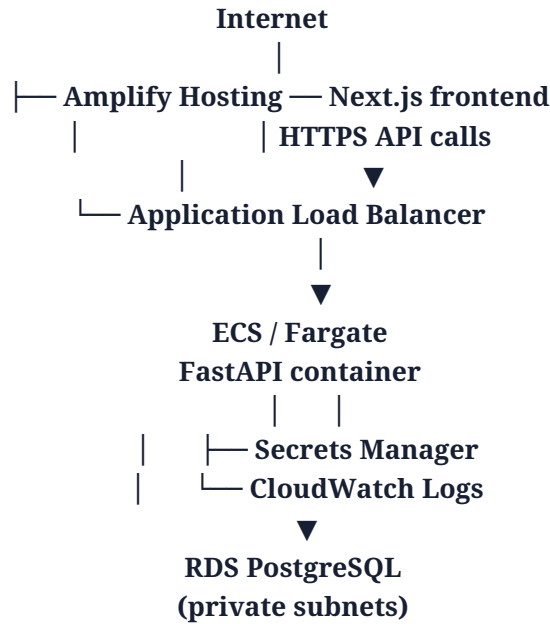
11. Testing strategy

Layer	Tool	Examples	Target
Backend unit	pytest	Preset coordinate generation; Spotify response normalization; username validation; authorization helpers.	Fast, isolated logic tests.
Backend API	pytest + httpx/TestClient	Public/private profile access; save mosaic; callback invalid state; 401/403/404 behavior.	Critical API contracts.
Database	pytest + test Postgres	Constraints, migrations, repository queries, transactions.	Relational correctness.
Frontend unit	Vitest + RTL	Profile form, AlbumTile, read-only state, search state.	Component behavior.
E2E	Playwright	Edit profile → create mosaic → publish → open public profile.	One golden-path smoke suite.
Security	dependency/container scans	Known dependency issues, secret detection, image scan.	Basic supply-chain hygiene.

12. Security and privacy

- OAuth state protection: random state value, short expiry, one-time use; reject mismatches.
- Server-side tokens: Spotify refresh token never goes to the browser. Encrypt at rest if stored in PostgreSQL; consider KMS-backed application encryption in the AWS phase.
- Application session: HttpOnly + Secure + SameSite cookie; rotate/delete on logout and account disconnect.
- Authorization: every write query scopes by the authenticated user; never authorize based on frontend-supplied user IDs.
- Secrets: Spotify client secret, DB password, encryption keys, and production session secret live in Secrets Manager, not .env committed to Git.
- Database networking: RDS private; only backend security group can reach port 5432.
- Logging: redact Authorization headers, cookies, tokens, email addresses where possible.
- Privacy controls: public/private profile, disconnect Spotify, delete local account/profile data.
- Public DTO minimization: expose only username/display name/bio/avatar URL/selected albums/layout needed for the profile.
- Policy review before launch: re-check Spotify Developer Policy and quota-mode requirements before expanding beyond a small beta because these rules can change.

13. AWS deployment architecture



Networking plan

- VPC across at least two Availability Zones.
- Public subnets: Application Load Balancer; NAT only if you choose to give private tasks outbound internet through NAT.
- Private application subnets: ECS tasks. FastAPI needs outbound HTTPS to Spotify; choose NAT Gateway for conventional architecture or evaluate a lower-cost design for a portfolio environment.
- Private database subnets: RDS PostgreSQL.
- ALB security group allows 443 from internet; ECS security group allows API port only from ALB security group; RDS security group allows 5432 only from ECS security group.

Cost note

A production-style VPC with NAT Gateway, ALB, ECS/Fargate, and RDS can cost real money even at low traffic. For the resume project, prioritize correctness but document a teardown/sleep strategy and AWS Budgets alarms. You can later compare a lower-cost deployment architecture while preserving the same application code.

14. CI/CD and Infrastructure as Code

Pull request workflow

12. Checkout repository.
13. Install Node and Python dependencies from lock files.
14. Run frontend ESLint, TypeScript type-check, and tests.
15. Run backend formatter/linter/type checks and pytest.
16. Build Docker image as a validation step.
17. Block merge if any required check fails.

Main branch deployment workflow

18. Authenticate to AWS with GitHub OIDC.
19. Build backend Docker image and tag with commit SHA.
20. Push image to ECR.
21. Register/update ECS task definition and deploy service.
22. Wait for ECS deployment health.
23. Amplify builds/deploys frontend from the connected main branch.

15. Observability and cost control

- FastAPI structured logs with request ID, route, status code, duration, and safe error category.
- CloudWatch alarms for ECS unhealthy tasks, ALB 5xx, RDS CPU/storage, and unusual error rates.
- Application metrics: owner profile saves, mosaic saves, public page loads — only events generated by your own app.
- Health endpoint should test process health; consider a separate readiness endpoint if database availability must be checked.
- AWS Budgets alert and a documented “pause/tear down” procedure after interviews/demo periods.
- Never log Spotify access/refresh tokens.

16. Milestones and suggested schedule

Week	Focus	Demo at end of week	Resume exposure
1	Repo, Next.js, FastAPI, Spotify OAuth	Connect Spotify → dashboard shows user.	Next.js, TypeScript, FastAPI, OAuth.
2	PostgreSQL + profile CRUD	Edit profile and share public read-only link.	PostgreSQL, SQLAlchemy, Alembic, authz.
3	Saved albums + mosaic builder	Choose albums, apply presets, persist layout.	Spotify API, drag/drop, data modeling.
4	Mobile polish + discover search + tests	Friend can open link; users can search local public profiles.	Responsive React, search, testing.
5	Docker + AWS deployment	Public HTTPS deployment on AWS.	Docker, ECR, ECS/Fargate, RDS, IAM.
6	CI/CD + Terraform + documentation	Push-to-deploy pipeline and reproducible infra.	GitHub Actions, IaC, Cloud architecture.

Schedule rule

Do not move to AWS before the complete MVP works locally with Docker Compose. A locally complete vertical slice is easier to debug and keeps cloud work from hiding application bugs.

17. Risks and mitigations

Risk	Impact	Mitigation
Spotify Development Mode user limit	Prevents open public onboarding for a portfolio build.	Treat project as owner + allowlisted beta users; public visitors can still view app-owned published data subject to your design and policy review. Re-check before launch.

		[1]
Spotify policy/API changes	An endpoint or allowed use can change.	Encapsulate Spotify calls behind SpotifyService; add source links in README and re-verify before demo/launch.
Derived “top albums” concept	Could conflict with policy and is not a Spotify endpoint.	Use user-selected Featured Albums from saved library instead. [3][6]
Drag/drop complexity	Can consume the project timeline.	Start with click-to-place and presets; add sophisticated drag/drop after persistence works.
AWS cost	Portfolio infrastructure may be expensive while idle.	Budgets alerts, small instances/tasks, short demo windows, and teardown plan.
Token/security bugs	Could expose account access.	Server-side tokens, HttpOnly session, redacted logs, security tests, least privilege.
Too much infrastructure too early	Delays the product users can see.	Finish phases 0–7 locally, then deploy.

18. Definition of done

- A new developer can clone the repository and run the application locally from the README.
- Owner can connect Spotify, create/edit a public profile, select saved albums, build/persist a mosaic, and publish it.
- A logged-out friend can open a read-only profile link on mobile and desktop.
- Album Mosaic search returns only public profiles from PostgreSQL.
- Automated tests cover critical auth, authorization, profile, mosaic, and search behavior.
- Production deployment is HTTPS, secrets are protected, database is private, and logs are available.
- CI runs on pull requests; production deployment is automated from main.
- Architecture, API, data model, setup, deployment, and tradeoffs are documented.
- GitHub repository contains screenshots/GIF, live demo link, and a concise architecture diagram.
- Resume bullets are specific to work actually implemented.

19. Resume and interview outcomes

Resume bullet drafts — use only after implemented

- Built a full-stack music-profile platform with Next.js/TypeScript and FastAPI, integrating Spotify OAuth 2.0 and saved-library APIs to create customizable, shareable album mosaics.
- Designed PostgreSQL data models and REST APIs for public/private profiles, session-based authorization, profile search, and persisted grid layouts using SQLAlchemy and Alembic.
- Containerized the Python API with Docker and deployed it on AWS ECS/Fargate behind an Application Load Balancer, with RDS PostgreSQL, Secrets Manager, CloudWatch, and an Amplify-hosted Next.js frontend.
- Implemented automated tests and GitHub Actions CI/CD, with infrastructure provisioned through Terraform and production secrets isolated from source control.

Interview stories this project should create

- Why split Next.js and FastAPI instead of using only Next.js API routes?
- How did you keep Spotify refresh tokens out of the browser?
- Why does your profile search use PostgreSQL instead of the Spotify API?
- How is a mosaic represented independently of screen size?
- How do you enforce read-only access for visitors?

- How did you handle OAuth failures and Spotify 429 responses?
- Why is RDS private and how do the AWS security groups connect?
- What tradeoff did you make between AWS architecture realism and cost?
- What does your CI/CD pipeline prove before a deployment?
- What would you change if the project needed 100,000 users?

20. Immediate next actions

Start here

The next coding milestone is intentionally tiny: Next.js “Connect Spotify” → FastAPI OAuth → Spotify GET /me → FastAPI session → Next.js dashboard showing the connected user. Do not build the mosaic editor until this vertical slice works.

24. Create the GitHub repository and folders frontend/, backend/, infrastructure/, docs/.
25. Bootstrap Next.js with TypeScript, App Router, Tailwind, ESLint.
26. Bootstrap FastAPI with GET /api/health and environment settings.
27. Create the Spotify developer application and register the local callback URL.
28. Implement OAuth start/callback plus GET /me; persist nothing except temporary state at first.
29. Add PostgreSQL/SQLAlchemy/Alembic only after the OAuth vertical slice succeeds.
30. Commit a screenshot/GIF at each milestone so the project history clearly shows progression.

21. Official references

- [1] Spotify — Quota modes — <https://developer.spotify.com/documentation/web-api/concepts/quota-modes>
- [2] Spotify — Get User's Top Items — <https://developer.spotify.com/documentation/web-api/reference/get-users-top-artists-and-tracks>
- [3] Spotify — Get User's Saved Albums — <https://developer.spotify.com/documentation/web-api/reference/get-users-saved-albums>
- [4] Spotify — Search for Item — <https://developer.spotify.com/documentation/web-api/reference/search>
- [5] Spotify — Get Current User's Profile — <https://developer.spotify.com/documentation/web-api/reference/get-current-users-profile>
- [6] Spotify — Developer Policy — <https://developer.spotify.com/policy>
- [7] Spotify — Web API Changelog, May 2026 — <https://developer.spotify.com/documentation/web-api/references/changes/may-2026>
- [8] AWS — Amplify support for Next.js — <https://docs.aws.amazon.com/amplify/latest/userguide/ssr-amplify-support.html>
- [9] AWS — Amazon ECS on AWS Fargate — <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/getting-started-fargate.html>
- [10] AWS — Amazon ECR Docker push — <https://docs.aws.amazon.com/AmazonECR/latest/userguide/docker-push-ecr-image.html>
- [11] AWS — Amazon RDS overview — <https://docs.aws.amazon.com/AmazonRDS/latest/gettingstartedguide/what-is-rds.html>
- [12] AWS — Create/connect RDS for PostgreSQL — https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_GettingStarted.CreatingConnecting.PostgreSQL.html
- [13] AWS — Secrets Manager — <https://docs.aws.amazon.com/secretsmanager/>
- [14] FastAPI — Dependencies / dependency injection — <https://fastapi.tiangolo.com/tutorial/dependencies/>
- [15] FastAPI — First steps and OpenAPI docs — <https://fastapi.tiangolo.com/tutorial/first-steps/>
- [16] Next.js — App Router — <https://nextjs.org/docs/app>
- [17] SQLAlchemy 2.0 — AsyncIO — <https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>
- [18] Alembic documentation — <https://alembic.sqlalchemy.org/en/latest/>
- [19] AWS — ECR private repository push — <https://docs.aws.amazon.com/AmazonECR/latest/userguide/docker-push-ecr-image.html>
- [20] AWS — Create a Secrets Manager secret — https://docs.aws.amazon.com/secretsmanager/latest/userguide/create_secret.html

Verification note: Spotify and cloud platform APIs/policies are changeable. Re-check the linked official documentation immediately before a production launch, major demo, or quota-extension request.